Chapter 2
VB 2010 Variables and Data Types

In algebra, a variable is a symbol, such as x and y, representing an unknown number or value. For example, we often use formula $d = v \bullet t$ to calculate the distance (d) traveled by an object moving at speed v over a time period t. In this formula, letters v, t, and d are variables. Once we know the values of variables v and t in the formula, we can derive a result of the formula. For example, if v = 65 miles / hour and t = 2 hours, then $d = v \times t$ = 65 x 2 = 130 miles.

Like algebra, we use variables to hold values in programming. This chapter first explains what variables are and demonstrates how to declare variables in VB programming. It further introduces data types, numeric operators and functions, as well as string operations. You will learn more about how to write programs to perform mathematical calculations with examples. The Chapter finally shows three types of possible errors in programming and introduces some VB programming skills.

1. Variables

Technically, a variable in a program is a piece of memory space set aside to hold a value of a certain type. In VB programming, you must declare a variable before you use it. To declare a variable is to set aside a piece of memory space in the computer. It is accomplished by using keywords **Dim** (standing for Dimension) and **As** with the following syntax:

```
Dim variableName [As <Data type>]
```

Where [As <Data type>] specifies the type of data that can be hold by the variable. Data types will be discussed in detail in next section. Examples of frequently used data types include Integer, Single or Double (precision floating point number), and String. By convention, a pair of square brackets in a syntax indicates that something included in it is optional, and a pair of angle brackets indicates that something included is required. Therefore, the above syntax shows, when you declare a variable, you may specify a data type. But once you use key word "As", you must provide a data type. Therefore, to declare a variable to hold a distance value, either one of the following two lines are valid.

```
Dim distance
```

or

```
Dim distance As Double
```

If you do not specify a data type to a variable, VB will automatically choose one for it when a value is first assigned to it. However, it is not a wise practice to let the computer to decide a data type for your variable because an automatically assigned data type may not be the most appropriate one. Moreover, without a known data type certain assistance related to the variable may not be available from IntelliSense. Therefore, you are encouraged to always explicitly specify data types when you declare variables!

In computer programming, we usually use words to name variables, which makes programs more readable than algebra formulas. The following are a few rules and conventions for naming variables in VB programming.

- A variable name must not be any of the VB reserved keywords such as Dim and As. For a complete list of VB reserved keywords, please refer to: http://msdn.microsoft.com/en-us/library/dd409611.aspx.
- Variable names must start with a letter. It may start with an underscore (_) symbol, but not other symbols or numbers.
- A variable name cannot contain spaces or certain characters including dot (.), plus sign (+), hyphens (-), slash (/), backward slash (\), asterisk (*), caret (^), ampersand (&), or other characters reserved as operators.
- A variable name may contain multiple words, in which case a naming convention is to use all lower case letters for the first word, and then capitalize the first letter of each following word. For example, *myCarSpeed*. Variable names should be succinct and very long variable names should be avoided.

Variables are not case sensitive in VB. When you declare a variable, you usually use a combination of lower and upper case letters in the name. When you type the variable in subsequent coding, you may ignore letter cases since IntelliSense can take care of it.

In VB, once you create a variable of a numeric type, i.e. a variable for holding numbers, a default value 0 is automatically stored in it. You can change the value in it at any time (this is why it is called a variable). In programming terms, to store a value in a variable is called to assign a value to the variable. For example,

```
Dim myCarSpeed As Double    ' at this point, myCarSpeed carries 0

myCarSpeed = 65             ' assigns value 65 to myCarSpeed
```

Value assignment takes place from right to left. In value assignment, units of values, such as speed unit mile per hour, should not be included in the value, because a variable (*myCarSpeed*) is declared to store one single value of a specified data type, e.g. double-precision floating number.

In VB 2010, one Dim statement may declare multiple variables. For example, you can declare two Double type variables x and y in one line.

```
Dim x, y As Double
```

Moreover, variable declaration and initial value assignment may be carried out by one line of code. For example:

```
Dim myCarSpeed As Double = 65
Dim travelTime As Double = 2
```

2. Data Types

In programming, data are not limited to numeric values. They can also be text strings, Boolean values, date and time, and so on. Moreover, numeric values can have various types such as short integer, long integer, single- or double-precision floating point number, etc. When you declare a variable, you often need to specify a data type to it so that the computer knows how much memory space to allocate and how to store values in the allocated memory space. On the other hand, a value of a certain data type can only be stored in variables designated for that data type. For example, a text string value, such as "Microsoft Visual Studio", can only be stored in a String type variable, and a double-precision value (e.g. 3.14159265359) can only be stored in a Double type variable. Many programming languages are very strict in data types. You cannot store a string value in a variable declared for integer data.

Different data types use different amount of memory space. A memory chip in the computer is made up of billions of micro transistors and capacitors. Each pair of transistor and capacitor is a **memory cell** that can be either charged (representing 1) or discharged (representing 0). A memory cell that holds 1 or 0 is called a **bit**. 8 bits make a **byte** (Figure 1). To store a piece of data such as a number or a text string in the computer memory, the data is converted into binary codes which are then stored in the memory cells.

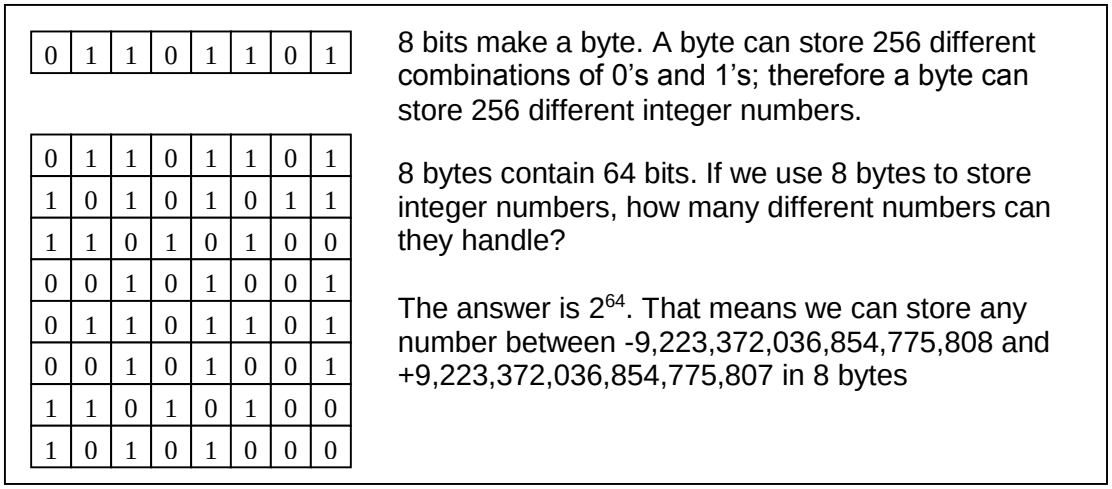


Figure 1 bits and bytes

Table 1 lists a few frequently used numeric data types of VB.NET with corresponding memory size and value ranges:

Table 1. Frequently used numeric data types of VB.NET

Data Type	Memory Size	Value Range
Integer	4 bytes (32 bits)	-2,147,483,648 to 2,147,483,647
Short	2 bytes (16 bits)	-32,768 to 32,767
Long	8 bytes (64 bits)	-9,223,372,036,854,775,808 to +9,223,372,036,854,775,807
Single	4 bytes (32 bits)	Negative: -3.4028235E+38 to -1.401298E-45

		Positive: 1.401298E-45 to 3.4028235E+38
Double	8 bytes (64 bits)	Negative: -1.79769313486231570E+308 to -4.94065645841246544E-324 Positive: 4.94065645841246544E-324 to 1.79769313486231570E+308

For information about all VB.NET data types, please refer to http://msdn.microsoft.com/en-us/library/47zceaw7.aspx

In addition to number of bits, different data types also differ in using the bits to store various component of numbers. For example, short integer (Short) uses 16 bits among which 1 bit is used for a sign (+ or -) and 15 bits are used for digits. Double-precision floating point number type (Double) uses 64 bits among which 1 bit is used for a sign, 11 bits are used to store an integer exponent, and 52 bits to store the significant decimal digits.

Different numeric data types have different value ranges and precision. When you declare variables in programming, you should use proper data types that preserve data precision while minimizing memory consumption and maximizing computation performance. For example, you may declare a short integer (2 bytes) variable for the number of students in a class, an integer (4 bytes) variable to store population of a state, and a long integer (8 bytes) variable to store unique IDs of features in a geodatabase. Single and Double floating-point data types have different number of significant digits. A Single precision floating point number can hold 8 significant digits and a double precision floating point number can handle as many as 18 significant digits. To store surface elevation in meters, the Single data type is sufficient because elevation values normally do not require extremely high precision (two or three decimal places should be good enough in most cases). But to store geographic coordinates (latitude and longitude), you have to use Double data type variables because they require more significant digits.

3. Arithmetic Operations

3.1 Arithmetic operators

Arithmetic operations are carried out in programming by using operators on numeric values or variables carrying numeric values. VB uses the following five basic arithmetic operators:

- Addition: +
- Subtraction: -
- Multiplication: *
- Division: /
- Exponentiation: ^

In VB 2010, a couple of less frequently used but useful operators are integer division (\) and modulus (mod) operators. Integer division is a division in which the fractional part is discarded and only the integer part is retained. For example,

14 \ 3 is 4 (14 divide by 3 is 4.6666….)

19 \ 5 is 3

Modulus operation finds the remainder when dividing one whole number by another. For example,

14 mod 3 is 2 (2 is the remainder of the division)
19 mod 5 is 4 (4 is the remainder of the division)

3.2 Incrementation

If we have a variable named *var*, we can increase its value by 1 using the following code.

```
var = var + 1
```

First, it calculates the right hand part, then assigns the result to variable *var*

Suppose *var* is initially 10, after executing the above code line, the value of *var* becomes 11. Likewise, we can increase the value of a variable by any number n.

```
var = var + n
```

In VB.NET, the above incrementation expressions are usually written as

```
var += 1      this does exactly same as var = var + 1
var += n      this does exactly same as var = var + n
```

3.3 Precedence and associativity

Like mathematics, arithmetic operators in VB have different precedence. The precedence of frequently used operators from high to low is as the following:

1) Exponentiation (^)
2) Unary identity and negation (+, -)
3) Multiplication and division (*, /)
4) Integer division (\)
5) Modulus arithmetic (Mod)
6) Addition and subtraction (+ -)

In computation, operations move from left to right in an expression when operators are of same precedence. Parentheses can be used to force some parts of an expression to be evaluated before others. This can override both the order of precedence and the left associativity. If nested parentheses are used, inner parentheses are processed prior to outer parentheses.

For example, given expression 5 + 4 ^ 2 - 3 * 2, the highest precedence operation is 4 ^ 2, resulting 16, and the second high precedence operation is 3 * 2, resulting 6. After evaluating these high precedence operations, the expression becomes 5 + 16 - 6. Finally, operations are performed from left to right, resulting 15.

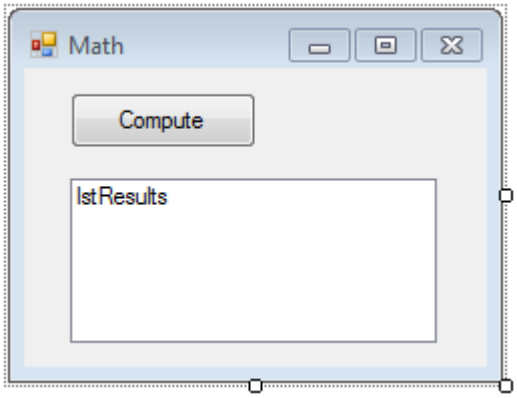
Given expression (15 - (3 + 2)) / (6 - 4), the innermost parentheses (3 + 2) is first evaluated, which converts the expression into (15 - 5) / (6 - 4). Then the two pairs of parentheses are evaluated from left to right, so that the expression is transformed to 10 / 2. Finally, the diversion is performed and result 5 is derived.

Example 1: Create a VB project to calculate results of the following math expressions and display results in a list box.

10 + 5 * 10 - 5
10 + 5 * (10 - 5)
12 - ((3 + 7) / 2 + 1)
3.14 * 4 ^ 2
((10 - 7) ^ 2 + (8 - 4) ^ 2) ^ 0.5

First, start Microsoft Visual Studio 2010 and create a new VB project using the Windows forms application template. Name your project "Arithmetic".

Second, design a project GUI as the screenshot below and set control properties as the following.



Control	Property Settings
Form	Name: frmMath; Text: Math Size: 242, 190
Button	Name: btnCompute Text: Compute
ListBox	Name: lstResults Size: 180, 82

Third, double click on the compute button to bring up the Click event procedure of the button control. For simplicity, exact parameters of the button click event procedures are omitted. This convention will be followed throughout the rest of the book.

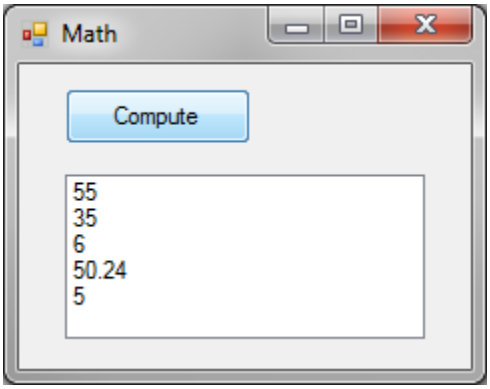
```
Private Sub btnCompute_Click(...) Handles btnCompute.Click

End Sub
```

Forth, write the following code inside the button's Click event procedure

```
Dim result1 As Double
Dim result2 As Double
Dim result3 As Double
Dim result4 As Double
Dim result5 As Double

result1 = 10 + 5 * 10 - 5
result2 = 10 + 5 * (10 - 5)
result3 = 12 - ((3 + 7) / 2 + 1)
```

```
result4 = 3.14 * 4 ^ 2
result5 = ((10 - 7) ^ 2 + (8 - 4) ^ 2) ^ 0.5

lstResults.Items.Add(result1)
lstResults.Items.Add(result2)
lstResults.Items.Add(result3)
lstResults.Items.Add(result4)
lstResults.Items.Add(result5)
```

The code above consists of three blocks. The first block declares five Double (precision floating point number) variables, the second block performs calculations and assigns result of each expression to a corresponding variable, and the last block displays all results in the list box. Now, start the program and click the button. You should see a Window similar to the following.



4. Built-in math functions

Simple arithmetic operations can be carried out by arithmetic operators. Complicated math operations are often carried out by math functions built in the programming language. A math function takes one or more input values, performs computation, and returns a result. The following are a few frequently used math functions provided by VB.NET.

- `Math.Sqrt(n)` – returns the square root of number n. For example,
  Math.Sqrt(9) returns 3
  Math.Sqrt(0) returns 0
- `Math.Abs(n)` – returns the absolute value of n. For example,
  Math.Abs(-10) returns 10
  Math.Abs(10) returns 10
- `Math.Round(n, r)` – rounds number n with r decimal places. If r is omitted, the function rounds number n to the nearest whole number. For example,
  Math.Round(2.7) returns 3
  Math.Round(3.14159, 2) returns 3.14
- `Math.Pow(a, n)` – powers value a by exponent n. For example,
  Math.Pow(5, 2) returns 25
  Math.Pow(2, -1) returns 0.5
- `Int(n)` – rounds number n to the lower whole number. For example,
  Int(2.7) returns 2

Int(3) returns 3
Int(-2.7) returns -3

For all VB.NET math functions, please refer to
http://msdn.microsoft.com/en-us/library/thc0a116.aspx.

Example 2: Create a new VB project named "Euclid" to calculate the Euclidean distance between two points. The main form contains one button named "btnCompute" and labeled "Compute". When the button is clicked, the distance between points (x1, y1) and (x2, y2) is displayed in a message box. Coordinates of the two points are as the following.

x1 = 5.25, y1 = 10.32
x2 = 8.64, y2 = 6.48

The Euclidean distance formula is $d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. To calculate the square root of a value in VB, you will use the built-in math function Math.Sqrt(). Please enter the following code in the compute button click event procedure to calculate distance.

```
Dim x1 As Double
Dim y1 As Double
Dim x2 As Double
Dim y2 As Double

x1 = 5.25
y1 = 10.32
x2 = 8.64
y2 = 6.48

Dim distance As Double
distance = Math.Sqrt((x1 - x2) ^ 2 + (y1 - y2) ^ 2)

MessageBox.Show(distance)
```

In the code above, the first block declares four Double type variables, *x*1, *y*1, *x*2, and *y*2, and the second block assigns values to each variable. Then it declares a Double type variable *distance* and calculates distance using the Euclidean distance formula. Finally, the result is reported by a message box.

5. Strings

5.1 String variables and string values

Besides numbers, another very frequently handled type of data in programming is string. A **string** is a sequence of characters treated as a single piece of data. Examples of strings include alphabetic letters, words, phrases, sentences, people's names, addresses, telephone numbers, and social security numbers. Variables declared as String type can only be used to store string values. For example, the following code declares two string variables *firstName* and *lastName*.

```
Dim firstName As String
Dim lastName as String
```

A String value must be quoted by a pair of double quotes. For example, “GIS”, “3.14”, and “Microsoft Visual Studio” are string values. However, variable names must not be quotes. If you quote a variable, for example “firstName”, it becomes a string constant. To assign string values to these variables, for example

```
firstName = "George"
lastName = "Washington"
```

Variable declaration and value assignment can be carried out within one line. For example,

```
Dim firstName As String = "George"
Dim lastName As String = "Washington"
```

5.2 String concatenation

Multiple string values can be combined into one value by string concatenation operators (& or +). For example,

```
Dim fullName As String = "George" & "Washington"
```

The value of variable *fullName* after executing the above line is “GeorgeWashington”. The concatenation operator can be used on string variables carrying values. For example,

```
Dim fullName As String = firstName & lastName
```

The resulting value of variable *fullName* will also be “GeorgeWashington” if variable firstName carries values “George” and variable lastName carries “Washington”. To separate the first name from the last name in the result, we can add a space character between the two name variables using the & operator.

```
Dim fullName As String = firstName & " " & lastName
```

Again, please keep in mind that string values, such as “George”, “Washington”, and the space character “ ”, each must be quoted in a pair of double quotes. However, variable names such as *firstName*, *lastName* and *fullName* must not be quoted. If you quote a variable, it is treated as a string value by the computer.

5.3 Using text boxes for input and output

In programming, we often use text boxes to receive user inputs and display computation results. To get a user input from a text box for processing, the content of a text box is often assigned to a variable which is processed subsequently. For example, we use the following code to read the user input in a text box control named *txtStudentName* and assign the value to a new string variable *strName*.

```
Dim strName As String = txtStudentName.Text
```

Code `txtStudentName.Text` is the Text property of the text box control, i.e. the text contained in the box. Whatever text the user enter in the text box control, it can be retrieved in programming through the Text property.

In addition to retrieving values from text box controls, we can also use code to update the content of text boxes, so that we can use text boxes to display computation results or information. To do so is to set a value to the Text property of text boxes. For example,

```
txtResult.Text = value
```

where *value* can be either a string value or a variable carrying a value such as a data processing result. This line displays a value to a text box named *txtResult*.

Please keep in mind that to display a value in a text box is to assign the value to the Text property of the text box. A common syntax error made by beginners is to directly assign a value a text box control instead of its Text property, like the following.

Wrong: `txtResult = value`

5.4 Conversion between strings and numbers

Because the Text property of a text box control is string type property, the content of a text box is a string!

It is fine when you assign the content of a text box to a string variable since the content in the text box is a string. However, we often need to use text boxes to take numeric user input, i.e. we often need to assign the contents of text boxes to numeric variables, for example,

```
Dim salary As Single          ' declare a variable of Single data type
salary = txtSalary.Text       ' assign user input in a text box to the variable
```

The code above normally works fine in VB as long as the user input is a valid number although the content of the text box (txtSalary.Text) is a string. If the user type in 60000.00 in the text box, the content in the text box is in fact a string “60000.00” instead of number 60000.00. When a string value “60000.00” is assigned to a numeric variable *salary*, a default setting of VB allows the language to automatically convert the string into a proper number. However, if the user enters “$60,000.00” in the text box, a run-time error will occur because VB cannot automatically convert string “$60,000.00” into a Single type number.

The default setting that allows VB to automatically convert values from one type to another can be turned off by placing the following statement in the top of the code module above all other code.

```
Option Strict On
```

Once this statement is in place, VB 2010 immediately becomes strict on data types in value assignment. As a result, code `salary = txtSalary.Text` will raise a syntax error because it assigns a string value to a numeric variable. In this case, we have to use an appropriate data type conversion function to explicitly convert the data into a proper type and assign the converted value to the variable.

The following are a few frequently used VB built-in functions that convert strings to numeric values

`CInt()`	converts string to Integer, e.g. CInt("65") returns 65
`CDbl()`	converts string to Double, e.g. CDbl("3.14159") returns 3.14159
`CSng()`	converts string to Single
`CDate()`	converts string to Date

The function that converts a number or other data type into a string is:

`CStr()`	e.g. CStr(3.14159) returns "3.14159"

> *In VB 2010, another way to convert a number to a string is to call the* `ToString()` *method of a numeric value object. For example,*
>
> ```
> Dim n As Integer = 100
> txtOutput.Text = n.ToString()
> ```

If statement "Option Strict On" is put in place, the following code is perfectly good:

```
Dim salary As Single = CSng(txtSalary.Text)
```

Conversely, to display a numeric computation result (say in Single or Double data type) in a text box, we can use

```
txtSalary.Text = CStr(result)
```

In addition to "specialized" data type conversion functions mentioned above, there is a "universal" VB data type conversion function named CType() with the following syntax:

```
CType(inValue, outDataType)
```

For example, we can use the CType function to carry out the following data conversions:

```
Dim age As Integer = CType(txtAge.Text, Integer)
Dim pi As Double = CType("3.1415926 ", Double)
```

Since 2002, Microsoft has provided a new universal data conversion object named `convert` in VB.NET. For example,

```
Dim age As Integer = Convert.ToInt32(txtAge.Text)
Dim salary As Double = convert.ToDouble(txtSalary.Text)
```

When a number is concatenated to a string, VB automatically converts the number to a string before concatenation. For example,

```
Dim licensePlate As String = "ABC" & 123
```

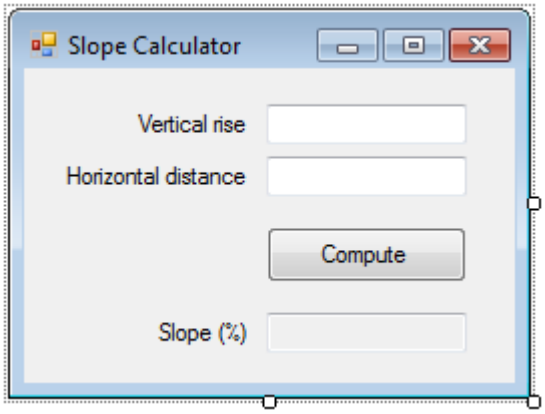
Where string constant “ABC” is concatenated with integer number 123. Strictly, number 123 should be converted to string in the above code line by a function, e.g. CStr(123). But it is usually not necessary in VB. After executing the above code line, variable licensePlate will carry value “ABC123”.

Example 3: Create a new VB project named “CalcSlopes” to calculate slope gradient vertical rise and horizontal distance. The GUI of the project will contain three text boxes and one button. The first text box will allow the user to enter an integer vertical rise in meters, the second text box will take an integer horizontal distance in meters, and the third text box will display the resulting slope gradient in percentage with two decimal places. Slope gradient is calculated by the following equation:

slope = vertical rise / horizontal distance * 100

Solution

To get started, please create a new VB project from the Windows forms application template and use the following specifications to design the GUI.



Control	Property Settings
Form	Neme: frmSlope Size: 260, 195
TextBox	Name: txtRise Size: 100, 20
TextBox	Name: txtDist Size: 100, 20
Button	Name: btnCompute Text: Compute; Size, 100, 20
TextBox	Name: txtSlope Size: 100, 20

Once you finish the GUI design, you can use the following code to implement the compute button’s click event procedure.

```
Private Sub btnCompute_Click(...) Handles btnCompute.Click
    ' declare some variable for use
    Dim rise As Integer
    Dim dist As Integer
    Dim slope As Double

    rise = CInt(txtRise.Text)    ' gets the vertical rise
    dist = CInt(txtDist.Text)    ' gets the horizontal distance
```

```
        slope = rise / dist * 100     ' computes slope

        ' display result
        txtSlope.Text = CStr(Math.Round(slope, 2))
    End Sub
```

The program first declares three variables of appropriate data types. Then it uses two lines to get the user input vertical rise and horizontal distance from the Text property of two text boxes, convert the string values into integers, and assign the integer values to corresponding variables, *rise* and *dist*. The third part performs the slope calculation. Since the quotient of two integer numbers is a double precision floating point number in VB, a Double data type variable *slope* is declared to store the calculation result. Finally, the last line assigns the resulting slope to the text box *txtSlope* for display. Before assigning the slope value to the text box, two VB built-in functions are used to process the result. First, function Math.Round() is used to round the slope with two decimal places. The function returns a value of the same data type as the input. Second, function CStr() converts the result of the first function in Double type to a String. In these function calls, the inner function is processed first.

In VB.NET programming, variable declaration and value assignment can be combined in one line. Therefore, the button's Click event procedure can be simplified as the following while it does the same job as the previous implementation.

```
    Private Sub btnCompute_Click(...) Handles btnCompute.Click
        Dim rise As Integer = CInt(txtRise.Text)
        Dim dist As Integer = CInt(txtDist.Text)

        Dim slope As Double = rise / dist * 100
        txtSlope.Text = CStr(Math.Round(slope, 2))
    End Sub
```

5.5 String properties and methods

In VB.NET, strings are objects. Like other software objects, strings also have properties and methods. A useful property of the string object is Length. This property tells how many characters a string contains. For example,

```
    Dim str As String = "Hello, World! "
    lstBox.Items.Add(str.Length)          ' 13 will be displayed in a list box
    stBox.Items.Add("3.1415926 ".Length)  ' 9 will be displayed in a list box
```

A method of an object performs a task. The following are some frequently used methods of string objects.

`ToUpper()`: converts all letters of a string to upper case
`ToLower()`: converts all letters of a string to lower case
`Trim()`: trims off blank spaces in the front and back of a string
`SubString(startIndex, length)`: extracts a portion from a string
`IndexOf(aSearchString)`: finds the starting position (index) of a search string in the current string. Returns -1 if the search string is not found.

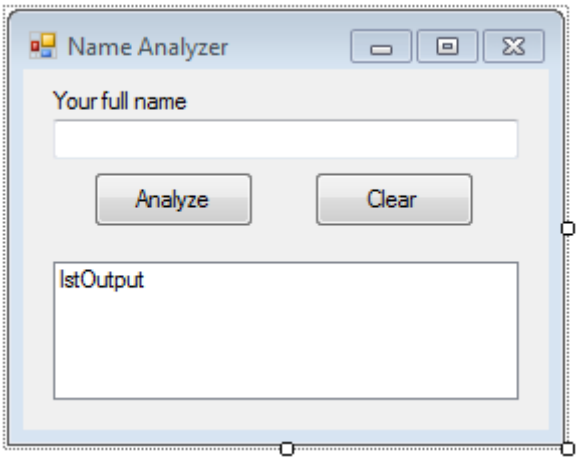
`LastIndexOf(aSearchString)`: finds the index of a search string in the current string by searching backwardly. Returns -1 if not found.

Examples of string methods,

```
Dim str As String = "Hello, World!"
str = str.ToUpper()     ' converts all letters of the string to upper case
lstBox.Items.Add(str)   ' the list box displays: HELLO, WORLD!

str = str.ToLower()     ' converts all letters of the string to lower case
lstBox.Items.Add(str)   ' the list box displays: hello, world!

str = "   VB 2010   "   ' assigns a new value to string variable str
str = str.Trim()        ' this removes all spaces around the string
                        ' so that variable str now carries "VB 2010" only

str = "abcdefghijk"     ' assigns a new value to string variable str
Dim s As String         ' declares another string variable s
s = str.SubString(2, 4) ' extracts 4 characters starting from the 3rd
                         ' Note: index always starts from 0 in VB.NET,
                         ' so that index 2 is the third character
lstBox.Items.Add(s)      ' the list box displays: cdef

lstBox.Items.Add(str.IndexOf("def"))    ' the list box displays: 3

str = "D:\VB2010\Files\Grades.txt"      ' assigns a file path to str
Dim p As Integer = str.LastIndexOf("\") ' finds the position of the
                                        ' last \ in the file path
lstBox.Items.Add(str.SubString(0, p))   ' displays the folder path only
lstBox.Items.Add(str.SubString(p))      ' displays the file name only
```

Example 4: Create a new VB project to analyze people's names. The project GUI contains a text box to allow the user to enter the full name of a person, a button to carry out the job, a list box to display result, and a button to clear the controls. When the user enters a full name of a person and clicks the analyze button, the program displays the first name, the last name as well as the length of the full name in the list box.

First, create a new VB project based on the Windows Forms Application template, and design the project GUI using the following specifications.



Control	Property Settings
Form	Name: frmNameAnalyzer Size: 278, 218 Text: Name Analyzer
TextBox	Name: txtFullName Size: 232, 20
Button	Name: btnAnalyze Size: 80, 28; Text: Analyze
Button	Name: btnClear Size: 80, 38; Text: Clear
ListBox	Name: lstOutput Size: 232, 69

Second, implement the analyze button's Click event as the following:

```
Private Sub btnAnalyze_Click(...) Handles btnAnalyze.Click
    ' declare some variable for use
    Dim fullName As String
    Dim length As Integer
    Dim space1 As Integer
    Dim space2 As Integer
    Dim firstName As String
    Dim lastName As String

    fullName = txtFullName.Text            ' gets the user input

    length = fullName.Length               ' gets length of the full name
    space1 = fullName.IndexOf(" ")         ' gets index of the first space
    space2 = fullName.LastIndexOf(" ")     ' gets index of the last space

    firstName = fullName.Substring(0, space1)  ' extracts the first name
    lastName = fullName.Substring(space2 + 1)  ' extracts the last name

    lstOutput.Items.Clear()    ' clears the list box
    ' display results in the list box
    lstOutput.Items.Add("First name: " & firstName)
    lstOutput.Items.Add("Last name: " & lastName)
    lstOutput.Items.Add("Full name length: " & Cstr(length))
End Sub
```

The procedure first declares six variables with intuitive names. Then it assigns the content of text box *txtFullName* to variable *fullName* (`fullName = txtFullName.Text`). With variable *fullName* carrying a user input value, the program gets the length of the string object from its Length property and assigns the value to variable *length* (`length = fullName.Length`).

To get the first name and last name from a full name, the program needs to know the position (index) of the first and last space characters in the full name. This task was carried out by calling methods `IndexOf(" ")` and `LastIndexOf(" ")` of a string object. Please note that space is enclosed in a pair of double quotation marks. The resulting

positions are assigned to variables space1 and space2. With the space positions known, method `Substring(index[, length])` is called twice to extract the first name and last name from the full name. This method takes one or two arguments. If you provide two arguments (index, length) to it, it extracts a sub string of a given length from a string by staring with the given index. If you only provide one argument (index) to the method, it extracts all the characters starting from the specified index to the end of the string. The results of the SubString() method are assigned to variables *firstName* and *lastName*.

The last block of the code first clears the list box by calling the `Clear()` method of the box, then displays all the results in the list box.

Finally, implement the clear button's Click event by using the following code:

```
Private Sub btnClear_Click(...) Handles btnClear.Click
    txtFullName.Text = ""
    lstOutput.Items.Clear()
    txtFullName.Focus()      ' sets focus on the input box
End Sub
```

The first line clears the text box by assigning a blank character to the text box, the second line clears the list box, and the last line calls the `Focus()` method of the text box control set focus on the box for new name entry.

When you test the program, make sure you provide a valid name containing at least two parts separated by space. The program will crash if no name is entered in the text box or a name is entered but it does not contain a space in the middle.

6. Three Types of Errors

Errors are almost unavoidable when you write programs. Please do not be surprised if you make an error when you program. Experienced programmers make errors. In Chapter 3, you will learn how to find errors, a process generally called to debug. In order to debug, it is important for you to know three categories of errors:

a. **Syntax errors** are grammatical errors, such as misspellings, incompleteness, or incorrect punctuations. For example,

`lstBox.Itms.Add (3)`	the word "Items" is misspelled
`lstBox.Items.Add (2+)`	"2+" in the parenthesis is an invalid expression
`Dim m; n As Integer`	the semicolon should be a comma

Anything that cannot be understood by the programming language's compiler is a syntax error. In VB 2010, most syntax errors are spotted by the code editor when they are entered. The VB code editor underlines the syntax error with a blue squiggly line and displays a description of the error when you hover the mouse cursor over the squiggly line.

b. **Run-time errors** are errors that occur when a program runs. They are illegal operations caused by invalid values encountered during program execution. They cannot be captured by the code editor or compiler. Typically, a run-time error occurs when a program tries to open a file but the file does not exist, or in a math operation the divisor turns out to be zero. For example, when you use the following code to compute the population density

```
density = population / area
```

if variable *area* is not assigned a value (so that it carries the default value 0) or if its value is miscalculated as 0 by another function, a run-time error will cause the program to crash.

A program containing run-time errors can pass compiling. When a program runs in debugging mode in VB IDE, a run-time error causes the program to stop at the crash spot and the code line containing the error is highlighted.

c. **Logical errors**: If a program runs without crashing but it does not produce a correct result, logical errors must exist. Logical errors are usually caused by wrong algorithms for a computation. For example, a logical error exists in the following code that calculates the average of two numbers

```
average = num1 + num2 / 2
```

because the correct formula is

```
average = (num1 + num2) / 2
```

A program containing logical errors can pass compiling and can run through without crashing. This is the most insidious type of error because it has no warning and is difficult to find. To detect logical errors, you may have to compare program outcomes with manually calculated results. If a program does not produce expected results, the only way to debug logical errors is to carefully examine every line of code. Sometimes, you can separately test each algorithm or function to make sure it does its job correctly. The VB IDE provides a suite of debugging tools that assist programmers to identify errors.

7. Additional techniques

7.1 Message box

You may use text boxes or list boxes report run-time information or display computation results. Another useful technique is to use the VB message box. A message box is a VB built-in object that can be used to display information during run-time. The syntax of the message box is:

```
MessageBox.Show(<message>[, caption][, other options])
```

Where Show() is a method of the MessageBox object. It requires a parameter *message* which is the information to display. A message must be a string or a variable carrying a string value. The Show() method has a number of optional parameter including *caption* which displays a caption on the header bar of the message box. For example,

```
MessageBox.Show("Have fun in VB programming! ", "Hello")
```

The above code displays a message box containing a message “Have fun in VB programming!” The caption of the message box is “Hello”.

In addition to the *message* and *caption* parameters, the Show() method also accepts a number of other optional parameters that customize the appearance of the box. Parameters must be separated by commas. When you write code in the VB IDE, the VB IntelliSense keeps providing suggestions to you for additional options such as one button or more buttons, icons on the message box, etc.

7.2 Comments

Writing comments is an important part of the programming work. Comments are text embedded in programs to explain the code you write. Comments are not compiled and executed. They help code readers including yourself to understand your code (you may forget your own code a few days later). In VB, a comment is headed by an apostrophe sign ('). On the keyboard it is the single quote mark key. By default, a comment displays in green color in the VB code editor.

Normally, we write comments above a line or block of code. For example,

```
' declare some variable for use
Dim firstName As String
Dim lastName As String
```

A quick and short comment is often placed behind a code line. For example,

```
firstName = fullName.Substring(0, space1)  ' extracts the first name
lastName = fullName.Substring(space2 + 1)  ' extracts the last name
```

7.3 Code line breaking (line continuation)

Sometimes, code lines may be too long to fit in the width of the code editor. To void frequently scrolling from side to side to see the code, we can break a long line into multiple lines. In VB, an underscore (_) proceeded by a space is used to break a line. For example,

```
Private Function ComputeDistance(ByVal x1 As Double, _
                                 ByVal y1 As Double, _
                                 ByVal x2 As Double, _
                                 ByVal y2 As Double) As Double

Dim sentence As String = _
    "Thousands of characters can be typed in a line of code."
```

Line breaking cannot occur inside a pair of quotes. Therefore, you cannot simply cut a long string value in the middle as the following:

```
Dim sentence As String = "Thousands of characters can _
                          be typed in a line of code."
```

To cut a long string value into multiple lines, you can break the long string into multiple strings, and then concatenate them using concatenation operators. For example,

```
Dim sentence As String = "Thousands of characters can " & _
                         "be typed in a line of code."
```

With VB 2010 and new editions, the underscore character for line breaking can be omitted where there is an ampersand, an arithmetic operator, or a comma. For example, the following are valid code in VB 2010(+).

```
Dim sentence As String = "Thousands of characters can " &
                         "be typed in a line of code."

Private Function ComputeDistance(ByVal x1 As Double,
                                 ByVal y1 As Double,
                                 ByVal x2 As Double,
                                 ByVal y2 As Double) As Double
```

Summary and Highlights

A variable is a piece of memory space set aside for holding a certain type values. We use Dim statements to declare variables. Although specifying a data type is optional in declaring a variable in VB, it is strongly suggested to do it.

Data type defines the type of data a variable can handle. Some frequently used data types include Integer, Short, Long, Single, Double, String, and Boolean. Different data types require different amount of memory space on the computer and store values in the memory in different styles.

Precedence and associativity of arithmetic operations are similar to rules of algebra. Precedence of operators from high to low is: Exponentiation (^), Unary identity and negation (+, -), Multiplication and division (*, /), Integer division (\), Modulus arithmetic (Mod), and Addition and subtraction (+ -). When parentheses are used, the inner parentheses have higher precedence than outer parentheses.

Complex math operations are performed by built-in functions. Some common functions include Math.Abs(), Math.Sqrt(), Math.Round(), and Math.Pow().

String values must be quoted by double quotation marks. Strings can be combined by the concatenation operator (&, or +). When you use text boxes to take user input or to display results, keep in mind that the Text property of a text box control is a string. Proper data type conversion is often needed in using text boxes for input and output. Functions for converting strings to numeric values include CInt(), CSng(), CDbl(), etc. The function to convert a numeric value to a string is CStr().

A string is an object in VB. A string object has properties and methods. Some useful properties and methods of string objects include Length, IndexOf(), LastIndexOf, Substring(), ToUpper(), and ToLower().

There are three types of possible errors in programming: syntax error, run-time error, and logical error. Syntax errors are grammatical errors that can be caught by VB IDE. Run-time errors are illegal operations caused by invalid values encountered during program execution. These errors can pass compilation but cause programs to crash. Logical errors are wrong algorithms or computation procedure. Logical errors cannot be detected by the IDE and can pass compilation, but programs with logical errors produce incorrect results.

The Show() method of a message box object has one required parameter which is the message text. It has a series of optional arguments among which a caption is frequently used. When you write code, the IntelliSense keeps providing suggestions to you.

Writing comments is a necessary job in programming. A comment is a text message embedded in programs to explain the code. It is headed by an apostrophe symbol (') and appears in green text in the VB code editor by default. Comments are not compiled and executed.

Exercises

1. Answer the following questions.
   1) What are variables?
   2) How do you declare a variable?
   3) What happens to the computer when you declare a variable?
   4) How do you assign a value to a variable?
   5) What is data type?
   6) What are the differences between the Single and the Double data types?
   7) What are the differences between the Integer and Long data type?
   8) What to calculate the square root of a number?
   9) What does function Int() do?
   10) What does Math.Round() do?
   11) How do you concatenate strings?
   12) How do you convert values from one data type to another?
   13) What do these functions do? CStr, CInt(), CDbl(), CSng(), and CDate().
   14) How do you get length of a string?
   15) How do you convert a string to all upper or lower case letters?
   16) How do you remove all blank spaces in front and back of a string?
   17) How do you extract a portion from a string?
   18) How do you find the index (location) of a string in another string?
   19) What is a syntax error?
   20) What is a run-time error?
   21) What is a logical error?
   22) What does statement “Option Strict On” do?

2. Tell the result of the following arithmetic expressions without using programming.
   1) 3 + 2 * 5 - 6
   2) 3 + 2 * (5 - 6)
   3) (10 - (3 - 5)) / 2 ^ 2
   4) (10 - 3 - 5)) / 2 + 2
   5) 15 - 1 ^ (10 - 3) - 10
   6) 18 - (2 ^ (-3 + 5) - 6)
   7) (5 ^ 2 - 10) / (16 - 2 * (6 - 3) - 5)
   8) 1 - (5 * 5) ^ (3 / 6)
   9) 12 + (-18) / (-6) / (5 - 2)
   10) 100 / (100 - (100 - (100 - 50)))

3. Tell the result of the following expressions without using programming.
   Given a = 10, b = 5
   1) 2 * a - a + b
   2) 2 * a - (a + b)
   3) (a ^ 2 - b * 2) / (a + b * 2)
   4) a * b ^ (a - b * 2)
   5) b / (b - (b - 5))

   Given n = 2, x = 3, y = 4, z = 5
   6) x ^ n + y ^ n - z ^ n
   7) (z - x) ^ x / y ^ n
   8) (z - (x - y)) / n + n

9) (2 * x + y) / (x - z) / (y - n)
10) (2 * x + y) / ((x - z) / (y - n))

4. Analyze the following VB code and tell execution result displayed in the list box.

1)
```
Dim fName As String = "Barack"
Dim lName As String = "Obama"
lstOutput.Items.Add(fName & lName)
```

2)
```
Dim fName As String = "Barack"
Dim lName As String = "Obama"
Dim fullName As String = lName & ", " & fName
lstOutput.Items.Add(fullName)
```

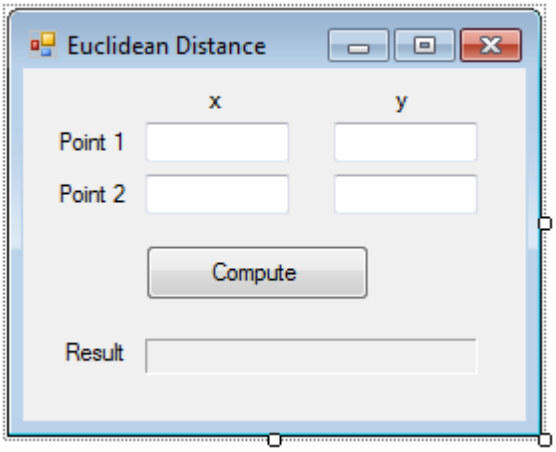
3)
```
Dim sideA As Double = 3.0
Dim sideB As Double = 4.0
Dim sideC As Double = Math.Sqrt(sideA ^ 2 + sideB ^ 2)
lstOutput.Items.Add("Side C of the triangle is " & CStr(sideC))
```

4)
```
Dim str As String = "Microsoft Visual Studio 2010"
lstOutput.Items.Add("'" & str & "' contains " & str.Length & "
characters.")
```

5)
```
Dim str As String = "Microsoft Visual Studio 2010"
Dim pos1 As Integer = str.IndexOf("o")
Dim pos2 As Integer = str.LastIndexOf("o")
lstOutput.Items.Add(str)
lstOutput.Items.Add("The first letter 'o' appears at index " & pos1)
lstOutput.Items.Add("The last letter 'o' appears at index " & pos2)
```

6)
```
Dim str As String = "Microsoft Visual Studio 2010"
Dim str1 = str.Substring(0, 5)
Dim str2 = str.Substring(17, 6)
lstOutput.Items.Add(str1 & str2)
```

5. Create a VB program named “EuclidDist” to compute the Euclidean distance between any two points. The program should use four text boxes to receive user input coordinates x1, y1, and x2, y2, a button to carry out the computation, and a label to display result. Refer to the following screenshot and property settings to design your application GUI.



Control	Property Settings
Form	Name: frmEuclid Text: Euclidean Distance Size: 268, 212
TextBox (4)	Name: txtX1, txtX2, txtY1, txtY2 TabIndex: 0, 1, 2, 3
Button	Name: btnCompute Text: Compute; TabIndex: 4
Label	Name: lblResult AutoSize: False BorderStyle: Fixed3D ForeColor: Red Size: 166, 18

6. In Exercise 3 of Chapter 1, you created a VB project named “Books”. Continue to develop on this project by adding a list box (name: lstBooks) into the main form. Instead of displaying individual pieces of book information by a series of message boxes, you will add the book information into the list box when the “Enter” button is clicked. Display the book information in the following format:

   Author, Year. Book title. Publisher. (Type)

   Tip: Before you modify the existing project, copy the entire solution folder into a different folder to preserve the original project.